

1 L2–3 Supervised Learning

1.1 Classification and Loss

ERM: population risk $\theta^* = \arg \min_{\theta} \mathbb{E}_{x \sim p} \text{err}(f(x; \theta), y(x))$; empirical risk $\hat{\theta} = \arg \min_{\theta} \frac{1}{N} \sum_i \text{err}(f(x_i; \theta), y_i)$. **Binary logistic**: $p(y = 1|x) = \sigma(w^\top x + b)$,

$$\ell = -y \log p - (1 - y) \log(1 - p), \quad \frac{\partial \ell}{\partial z} = p - y.$$

Multi-class: logits $z \in \mathbb{R}^K$, $p_k = e^{z_k} / \sum_i e^{z_i}$,

$$\ell(z, y) = -\log p_y = -z_y + \log \sum_j e^{z_j}, \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y].$$

One-hot CE: $-\sum_k y_k \log p_k$.

1.2 MLP and Backprop

Layer: $h^{(0)} = x$, $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$, $h^{(\ell)} = \phi(z^{(\ell)})$. Batch code convention often uses $X \in \mathbb{R}^{B \times n}$, $Z = XW + b$, $W \in \mathbb{R}^{n \times m}$. **Backprop**: for $y = f(x)$ with Jacobian $J = \partial y / \partial x$, chain rule gives $\nabla_x L = J^\top \nabla_y L$. Define $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$:

$$\partial_{W^{(\ell)}} L = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \partial_{b^{(\ell)}} L = \delta^{(\ell)},$$

$$\delta^{(\ell-1)} = ((W^{(\ell)})^\top \delta^{(\ell)}) \odot \phi'(z^{(\ell-1)}).$$

Activation: $\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$; sigmoid/ $\tanh$ saturate; ReLU $\max(0, z)$ keeps positive-side gradient; ELU/smooth variants improve negative side. **Subgradient**: ReLU at 0 uses any value in $[0, 1]$ by convention.

1.3 CNN

Motivation: locality + weight sharing. Convolution:

$$y_{i,j,\rho} = \sum_{u,v,c} K_{u,v,c,\rho} x_{i+u,j+v,c} + b_{\rho}.$$

For input $H \times W \times C_{\text{in}}$, kernel $K_h \times K_w$, padding P , stride S :

$$H' = \lfloor (H + 2P - K_h) / S \rfloor + 1, \quad W' = \lfloor (W + 2P - K_w) / S \rfloor + 1.$$

Params $K_h K_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$. **Scanning MLP**: same detector slides across space. **Pooling**: max/avg reduces spatial size and jitter. **Padding**: keeps boundary info; same padding for odd kernel uses $P = (K - 1) / 2$.

1.4 Optimization

GD/SGD: full-batch GD uses all data gradient; SGD uses one sample noisy gradient; mini-batch uses B samples to trade variance vs. throughput. $x_{k+1} = x_k - \eta_k \nabla f(x_k)$. L -smooth means gradient Lipschitz: $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$; if twice differentiable, $\|\nabla^2 f(x)\|_{\text{op}} \leq L$ (convex: $0 \leq \nabla^2 f(x) \leq LI$). Then

$$f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2} \|y - x\|^2,$$

Strong convexity: $f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2} \|y - x\|^2$; with L -smooth and twice differentiable, $\mu I \leq \nabla^2 f(x) \leq LI$. GD has linear convergence with $k = O(\kappa \log(1/\epsilon))$, $\kappa = L/\mu$. Thus $0 < \eta < 2/L$ decreases the smooth upper bound. **Newton**: from quadratic Taylor,

$$x_{k+1} = x_k - \eta [\nabla^2 f(x_k)]^{-1} \nabla f(x_k).$$

Momentum: $\Delta x^k = \beta \Delta x^{k-1} - \eta \nabla f(x^k)$, $x^{k+1} = x^k + \Delta x^k$. **Nesterov**: compute gradient at lookahead $x_k + \beta(x_k - x_{k-1})$. **Adaptive LR**: rescale each coordinate, helping ill-conditioned optimization. **AdaGrad**: $s_k = s_{k-1} + s_{k,i}^2$, $x_{k+1} = x_k - \eta s_k / (\sqrt{s_{k,i}} + \epsilon)$; rarely updated dims have smaller $s_{k,i}$ and larger effective LR; may decay too fast. **RMSProp**: $s_k = \rho s_{k-1} + (1 - \rho) s_{k,i}^2$. **Adam**:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) s_{k,i}, \quad v_k = \beta_2 v_{k-1} + (1 - \beta_2) s_{k,i}^2,$$

$$x_{k+1} = x_k - \eta \hat{m}_k / (\sqrt{\hat{v}_k} + \epsilon), \quad \hat{m}_k = m_k / (1 - \beta_1^k).$$

1.5 Regularization and Architectures

Dropout: randomly zero units, prevents co-adaptation/reliance on particular neurons or weights, approximates ensemble/implicit regularization (often L2-like). Scaling: inverted dropout divides by keep prob α at train and inference unchanged; standard dropout leaves train unchanged and multiplies activations by α at inference. **Early stopping**: monitor held-out validation; stop when training improves but validation worsens. **Weight decay**: L2 regularization stabilizes weights, $L_{\text{reg}}(w) = L_{\text{loss}}(w) + \alpha \|w\|_2^2$. **PCA color jitter**: add $P(\alpha \odot \lambda)$ to RGB channels, $\alpha_i \sim \mathcal{N}(0, 0.1)$, simulating lighting/color changes. **Gradient clipping**: $g \leftarrow g \min(1, \tau / \|g\|)$. **BatchNorm**: $\mu_B = \frac{1}{B} \sum_i z_i$, $\sigma_B^2 = \frac{1}{B} \sum_i (z_i - \mu_B)^2$,

$$\hat{z}_i = \gamma \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta.$$

BN backward: if $u_i = (z_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$, then $\frac{\partial L}{\partial z_i} = \frac{\partial L}{\partial u_i} \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} +$

$\frac{\partial L}{\partial \sigma_B^2} \frac{2(z_i - \mu_B)}{B} + \frac{\partial L}{\partial \mu_B} \frac{1}{B}$. **BN inference**: no mini-batch training distribution; use running stats $\mu_{\text{run}} \approx \frac{1}{N_{\text{batch}}} \sum_b \mu_B^{(b)}$, $\sigma_{\text{run}}^2 \approx \frac{1}{N_{\text{batch}}} \sum_b \frac{B-1}{B} \sigma_B^{2(b)}$. **LayerNorm**: normalize hidden dimension inside one sample; better for sequence models. **ResNet**: $h_{\ell+1} = h_\ell + F(h_\ell)$, residual path eases optimization. **DenseNet**: concatenate previous features. **FCN**: replace fully connected layer by convolution for dense prediction.

2 L4 Energy-Based Models

2.1 Energy View

Generative modeling: learn $P(X)$ or $P(X, y) = P(y)P(X|y)$ instead of $P(y|X)$. **EBM**: unnormalized score via energy

$$p_{\theta}(x) = \frac{e^{-E_{\theta}(x)}}{Z_{\theta}}, \quad Z_{\theta} = \int e^{-E_{\theta}(x)} dx.$$

Low energy means high probability; training and inference both need sampling or partition-function gradients.

2.2 Hopfield Network

State $y_i \in \{-1, 1\}$, local field $h_i = \sum_j \#i j w_{ij} y_j + b_i$. Update flips if $y_i h_i < 0$.

$$E(y) = -\sum_{i < j} w_{ij} y_i y_j - \sum_i b_i y_i = -\frac{1}{2} y^\top W y - b^\top y.$$

Asynchronous flip changes energy by $\Delta E = 2y_i h_i$; if $y_i h_i < 0$, energy decreases. Finite states imply convergence to a local minimum. **Hebbian storage**: for K patterns, $w_{ij} = \frac{1}{K} \sum_{p=1}^K y_i^p y_j^p$, $w_{ii} = 0$; stability sufficient condition $K \leq N / (4 \log N)$ for random patterns. Multiple patterns cause **spurious minima**. **Optimization view**: make desired patterns valleys and undesired states high energy. For $E(y) = -\frac{1}{2} y^\top W y$, objective $\min_W \sum_{y \in P} E(y) - \sum_{y' \notin P} E(y')$ gives contrastive update $W \leftarrow W + \eta (\mathbb{E}_{y \in P} [y y^\top] - \mathbb{E}_{y' \notin P} [y' y'^\top])$; cannot enumerate exponential state space, so sample $y \in P$, initialize y' from desired pattern, run 2–4 dynamics steps, update with $yy^\top - y'y'^\top$.

2.3 Boltzmann Machine

Stochastic Hopfield: $p(y) \propto e^{-E(y)}$. Conditional Gibbs update for $\{-1, 1\}$ units:

$$h_i = \sum_j w_{ij} y_j + b_i, \quad \log \frac{P(y_i = 1|y_{-i})}{P(y_i = -1|y_{-i})} = 2h_i, \quad P(y_i = 1|y_{-i}) = \sigma(2h_i).$$

Stochastic evolution: initialize $y^{(0)}$, Gibbs-sample coordinates $y_i(t+1) \sim \text{Bernoulli}(\sigma(z_i(t)))$ for $P(y_i = 1)$; recover stored pattern from tail samples by $\hat{y}_i = \mathbf{1}[\frac{1}{M} \sum_{t=L-M+1}^L y_i(t) > 0]$ (majority/sign). **Annealing**: $p_T(y) \propto e^{-E(y)/T}$, sample $y_i(T) \sim \text{Bernoulli}(\sigma(z_i(T)/T))$, then $T \leftarrow \alpha T$; lower T shifts exploration to exploitation. **BM MLE**: for $E(y) = -\frac{1}{2} y^\top W y$,

$$P(y) = \frac{e^{\frac{1}{2} y^\top W y}}{\sum_{y'} e^{\frac{1}{2} y'^\top W y'}}, \quad L(W) = \frac{1}{N P} \sum_{y \in P} \frac{1}{2} y^\top W y - \log \sum_{y'} e^{\frac{1}{2} y'^\top W y'}.$$

$$\begin{aligned} \nabla w_{ij} L &= \frac{1}{N P} \sum_{y \in P} y_i y_j - \mathbb{E}_{P_W} [y_i y_j] \\ &\approx \frac{1}{N P} \sum_{y \in P} y_i y_j - \frac{1}{M} \sum_{y' \in S} y'_i y'_j, \quad w_{ij} \leftarrow w_{ij} + \eta \nabla w_{ij} L. \end{aligned}$$

Learning = **positive phase** data correlation minus **negative phase** model correlation. With hidden h , condition on visible data v and average over h v :

$$\nabla w_{ij} L(W) = \frac{1}{|P|} \sum_{v \in P} \mathbb{E}_{h|v} [y_i y_j] - \mathbb{E}_{y' \sim P_W} [y'_i y'_j], \quad w_{ij} \leftarrow w_{ij} + \eta \nabla w_{ij} L.$$

2.4 RBM and Contrastive Divergence

RBM: visible v and hidden h form a bipartite graph; no within-layer edges:

$$E(v, h) = -v^\top W h - b^\top v - c^\top h.$$

Layer-wise Gibbs updates factorize:

$$z_j^h = c_j + \sum_i W_{ij} v_i, \quad P(h_j = 1 | v) = \sigma(z_j^h),$$

$$z_i^v = b_i + \sum_j W_{ij} h_j, \quad P(v_i = 1 | h) = \sigma(z_i^v).$$

Iterative sampling: alternate $h \sim P(h|v)$ and $v \sim P(v|h)$; after mixing, $(v^{(\infty)}, h^{(\infty)}) \sim P_W$. With K hidden samples per data and M free samples:

$$\hat{\nabla} W_{ij} L(W) = \frac{1}{N P K} \sum_{v \in P} \sum_{k=1}^K v_i^{(0)} h_{j,k}^{(0)} - \frac{1}{M} \sum_{m=1}^M v_{i,m}^{(\infty)} h_{j,m}^{(\infty)}.$$

CD-1 replaces the long negative chain by one reconstruction:

$$\nabla W_{ij} L \approx \frac{1}{N P} \sum_{v \in P} \left(v_i^{(0)} h_j^{(0)} - v_i^{(1)} h_j^{(1)} \right).$$

2.5 Sampling

Inverse CDF: sample $u \sim U[0, 1]$, choose category by cumulative mass. **Box-Muller**: Gaussian from uniform:

$$z_1 = \sqrt{-2 \log u_1} \cos(2\pi u_2), \quad z_2 = \sqrt{-2 \log u_1} \sin(2\pi u_2).$$

Importance sampling: $\mathbb{E}_p[f(x)] = \mathbb{E}_q[f(x)p(x)/q(x)]$. Good q must cover p 's high-mass region. **MCMC**: Markov chain with stationary π . Detailed balance $\pi(x)T(x \rightarrow x') = \pi(x')T(x' \rightarrow x)$ implies stationarity. Uniform minorization

$$\min_{z,z'} \frac{T(z \rightarrow z')}{\pi(z')} > 0$$

is a strong sufficient condition for uniform ergodicity. **MH**: proposal $q(x'|x)$, accept

$$\alpha = \min \left(1, \frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)} \right).$$

For EBM, partition Z cancels. **Gibbs**: update one conditional exactly; as MH, use $q(s_i \rightarrow s'_i) = p(s'_i | s_{j \neq i})$, so accept prob. = 1. **SGLD**: $x_{t+1} = x_t - \frac{\eta}{2} \nabla_x E_{\theta}(x_t) + \sqrt{\eta} \xi_t$.

3 L5 Generative Adversarial Networks

3.1 Implicit Generative Model

Instead of density $p_{\theta}(x)$, learn sampler $x = G_{\theta}(z)$, $z \sim p_z$. Need differentiable distance between generated and real distributions; GAN learns this distance with a discriminator.

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z \sim p_z} \log(1 - D(G(z))).$$

For fixed G :

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substitute:

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_g).$$

Non-saturating generator loss $-\mathbb{E}_z \log D(G(z))$ gives stronger gradients.

3.2 Evaluation and Failure Modes

Mode collapse: generator covers few modes. **Sample diversity**: for multi-modal p , forward KL is mode-covering, while reverse KL and JSD can be mode-seeking/collapse:

$$\text{KL}(p \| q) = \mathbb{E}_p \log \frac{p}{q}, \quad \text{KL}(q \| p) = \mathbb{E}_q \log \frac{q}{p},$$

$$\text{JSD}(p \| q) = \frac{1}{2} \text{KL}(p \| m) + \frac{1}{2} \text{KL}(q \| m), \quad m = \frac{p+q}{2}.$$

Training instability: moving target game can oscillate; if D 's task is too simple/strong, G gets vanishing or weak gradients. **Inception Score** uses classifier $f(y|x)$:

$$\text{IS} = \exp \left(\mathbb{E}_{x \sim G} \text{KL}(f(y|x) \| p_f(y)) \right), \quad p_f(y) = \mathbb{E}_{x \sim G} [f(y|x)].$$

Since $\mathbb{E}_x \text{KL}(f(\cdot|x) \| p_f) = -\mathbb{E}_x H(f(\cdot|x)) + H(p_f)$, high IS needs confident images (low conditional entropy) and diverse classes (high marginal entropy). IS never uses p_{data} , so it cannot detect memorization and is blind to intra-class artifacts. **FID**: use Inception-v3 pool3 2048-d features; estimate Gaussian statistics (μ_p, Σ_p) for p_{data} and (μ_G, Σ_G) for p_G :

$$\text{FID} = \|\mu_p - \mu_G\|_2^2 + \text{tr} \left(\Sigma_p + \Sigma_G - 2(\Sigma_p \Sigma_G)^{1/2} \right).$$

Lower FID is better.

3.3 GAN Training Techniques

Feature Matching: train G to match statistics of discriminator features instead of only fooling D :

$$L_G(\theta) = \left\| \mathbb{E}_{\hat{x} \sim G_{\theta}} [f(\hat{x}; \phi)] - \mathbb{E}_{x \sim p_{\text{data}}} [f(x; \phi)] \right\|_2^2.$$

Here $f(x; \phi)$ is an internal feature of D , often the final activation layer before classification. **Other tricks**: One-Sided Label Smoothing; Historical Averaging; Minibatch Discrimination; Virtual Batch Normalization.

3.4 WGAN Family

JSD can be flat when supports do not overlap. Wasserstein-1:

$$W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p} f(x) - \mathbb{E}_{x \sim q} f(x).$$

WGAN objective:

$$\min_G \max_{D: \|D\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} [D(x)] - \mathbb{E}_{\hat{x} \sim p_G} [D(\hat{x})].$$

Lipschitz constraint: weight clipping, spectral norm, or WGAN-GP. For WGAN-GP, sample $\hat{x} = (1 - \epsilon)x + \epsilon \hat{x}$, $\epsilon \sim U(0, 1)$, between real x and fake $\hat{x}$:

$$\max_D \mathbb{E}_{x \sim p_{\text{data}}} D(x) - \mathbb{E}_{\hat{x} \sim p_G} D(\hat{x}) - \lambda \mathbb{E}_{\hat{x}} (\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2.$$

DCGAN: conv/deconv, batchnorm, stable architecture. **BigGAN**: scale + class conditioning + truncation trick. **GigaGAN**: text-to-image GAN scaling. **R3GAN**: regularized robust objective.

3.5 Variants and Applications

Conditional GAN: $G(z, c)$, $D(x, c)$. **Semi-supervised GAN**: discriminator has $K + 1$ classes, K real labels plus fake. **BigAN/ALI**: learn encoder $E(x)$ with discriminator over (x, z) pairs; gives representation learning. **Pix2Pix**: paired image-to-image translation with conditional GAN + reconstruction loss. **CycleGAN**: unpaired translation using cycle consistency

$$G: X \rightarrow Y, \quad F: Y \rightarrow X, \quad \|F(G(x)) - x\| + \|G(F(y)) - y\|.$$

DragGAN: interactive image manipulation by optimizing latent/code with handle-point constraints.

4 L6 Flow and VAE

4.1 Normalizing Flow

Invertible differentiable $x = G_\theta(z)$:

$$\log p_X(x) = \log p_Z(G^{-1}(x)) + \log \left| \det \frac{\partial G^{-1}}{\partial x} \right|.$$

Composition $G = f_K \circ \dots \circ f_1$ adds log determinants. Flow design needs invertible layer, efficient inverse, efficient determinant. **Coupling**: split $x = (x_a, x_b)$:

$$y_a = x_a, \quad y_b = x_b \odot e^{s(x_a)} + t(x_a), \quad \log |\det J| = \sum s(x_a).$$

Flow: invertible 1×1 convolution mixes channels; LU decomposition makes determinant cheap: $\log |\det W| = \sum_i \log |u_{ii}|$. **Dequantization**: add uniform noise to discrete pixels so continuous density is well-defined. **Continuous NF / Neural ODE**: $\frac{dz(t)}{dt} = f_\theta(z(t), t)$,

$$\log p(z(t_1)) = \log p(z(t_0)) - \int_{t_0}^{t_1} \text{tr} \left(\frac{\partial f_\theta}{\partial z(t)} \right) dt.$$

4.2 EM and Variational Inference

Latent-variable likelihood $\log p_\theta(x) = \log \sum_z p_\theta(x, z)$. Jensen:

$$\log p_\theta(x) = \log \mathbb{E}_q \frac{p_\theta(x, z)}{q(z)} \geq \mathbb{E}_q \log p_\theta(x, z) + H(q) = \mathcal{L}(q, \theta).$$

Gap:

$$\log p_\theta(x) - \mathcal{L}(q, \theta) = \text{KL}(q(z) \| p_\theta(z|x)).$$

EM: E-step $q(z) = p_{\text{old}}(z|x)$, M-step maximize $\mathbb{E}_q \log p_\theta(x, z)$.

4.3 VAE

Encoder $q_\phi(z|x)$, decoder $p_\theta(x|z)$, prior $p(z)$. ELBO:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)).$$

Reparameterization:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Diagonal Gaussian KL:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2 I) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1).$$

Terms: reconstruction = data fidelity; KL = latent regularization. **Pixel-VAE/NVAE**: hierarchical latent variables improve global coherence and likelihood.

5 L7 Diffusion and Flow Matching

5.1 DDPM

Forward:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_{t-1}, \beta_t I), \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

Closed form:

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I), \quad x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon.$$

Posterior $q(x_{t-1}|x_t, x_0) = \mathcal{N}(\bar{\mu}_t, \bar{\beta}_t I)$:

$$\bar{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t, \quad \bar{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Predict noise:

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}, \quad L_{\text{simple}} = \mathbb{E} \|\epsilon - \epsilon_\theta(x_t, t)\|^2.$$

Sampling:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z.$$

5.2 Score-Based Models

Score: $s_t(x) = \nabla_x \log p_t(x)$. Denoising score matching avoids unknown data score by using noisy conditional score:

$$\nabla_{x_t} \log q(x_t|x_0) = - \frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t}.$$

SDE:

$$dx = f(x, t)dt + g(t)dW_t, \quad dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)]dt + g(t)d\tilde{W}_t.$$

Probability-flow ODE:

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt.$$

Likelihood along ODE:

$$\log p_0(x_0) = \log p_T(x_T) + \int_0^T \text{tr} \left(\frac{\partial f_\theta}{\partial x_t} \right) dt.$$

Annealed Langevin: use multiple noise levels; high noise gives global movement, low noise refines detail.

5.3 DPM-Solver and Applications

DPM-Solver integrates diffusion ODE with high-order exponential integrators, reducing sampling steps. **Latent diffusion**: diffuse in autoencoder latent space, cheaper than pixel space. **DiT/SORA**: transformer over spacetime latent patches. **DALL-E 2**: CLIP prior + diffusion decoder. **DALL-E 3**: synthetic captions improve prompt following. **SDEdit**: add noise to input image then denoise under prompt. **ControlNet**: extra condition branch injects edges/depth/pose. **Logit-normal sampling**: biases timestep sampling toward informative middle region.

5.4 Flow Matching

Continuous flow $dx_t/dt = v_\theta(x_t, t)$. Objective:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \|v_\theta(x_t, t) - u_t(x_t)\|^2.$$

Conditional path from noise x_0 to data x_1 : $x_t = (1 - t)x_0 + tx_1$, target $u_t = x_1 - x_0$. Conditional Flow Matching (CFM) samples pairs and regresses conditional velocity; marginal vector field is recovered in expectation. **Non-uniqueness**: many vector fields induce same marginals; choice affects training variance and sampling trajectory. **SD3**: flow target often parameterizes velocity between noise/data under continuous timestep.

6 L8–9 Sequence Modeling

6.1 Sequence Data and Tokenization

Sequence data includes language, audio, action; images/DNA/protein can be serialized. Autoregressive factorization:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t|x_{<t}).$$

Tokenization: text $\rightarrow$ basic units $\rightarrow$ token IDs. Byte-Pair Encoding (BPE) repeatedly merges frequent adjacent symbols; balances vocabulary size and sequence length. **Embedding**: token ID $i \rightarrow e_i = W_E[i]$; positional info is needed because token embeddings alone are order-free.

6.2 Autoregressive CNN and Pixel Models

WaveNet: causal/dilated temporal convolution expands receptive field without recurrence. **PixelCNN**: image likelihood

$$p(x) = \prod_i p(x_i|x_{<i})$$

under raster order; masked convolution prevents access to future pixels. Gated PixelCNN uses gates to improve expressiveness.

6.3 RNN and LSTM

RNN:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad y_t = W_{hy}h_t + b_y.$$

BPTT multiplies many Jacobians; singular values < 1 vanish, > 1 explode. **LSTM**:

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \quad i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i),$$

$$\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t).$$

Additive cell path gives controllable memory; forget gate decides retention.

6.4 Attention and Transformer

For token t : $q_t = W_Qx_t$, $k_j = W_Kx_j$, $v_j = W_Vx_j$.

$$\alpha_{tj} = \frac{\exp(q_t^\top k_j / \sqrt{d_k})}{\sum_i \exp(q_t^\top k_i / \sqrt{d_k})}, \quad y_t = \sum_j \alpha_{tj} v_j.$$

Matrix form:

$$\text{Attn}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V.$$

Multi-head: parallel attention subspaces, concatenate and project. **Causal mask**: block future tokens. **Transformer block**: attention + MLP + residual + normalization. Complexity $O(T^2d)$; FlashAttention reduces memory IO, Mamba/SSM explores linear-time sequence mixing.

6.5 Objectives and Case Studies

Objectives: autoregressive LM, masked/denoising, contrastive predictive. **Recipe**: pretraining, fine-tuning, low-rank tuning. **GPT**: decoder-only causal Transformer; next-token pretraining gives in-context learning with scale. **ViT**: image patches as tokens. **Swin**: shifted local windows for hierarchical vision. **CLIP**: contrastive image-text alignment,

$$L = -\frac{1}{2} \left[\frac{1}{N} \sum_i \log \frac{e^{\epsilon_{ii}^\top / \tau}}{\sum_j e^{s_{ij}^\top / \tau}} + \frac{1}{N} \sum_i \log \frac{e^{\epsilon_{ii}^\top / \tau}}{\sum_j e^{s_{ji}^\top / \tau}} \right].$$

SigLIP: pairwise sigmoid loss avoids global softmax. **Whisper**: audio Transformer trained on large-scale speech. **MAR**: masked autoregressive modeling for images.

7 L10 Modern Large Models

7.1 Pretraining and Scaling

Autoregressive LM: maximize $\sum_t \log p_\theta(x_t|x_{<t})$. Base model learns continuation; chat model learns instruction-following behavior through post-training. **Scaling laws**: loss decreases smoothly with model size N , data D , compute C , often power-law:

$$L(N) \approx L_\infty + aN^{-\alpha}, \quad L(D) \approx L_\infty + bD^{-\beta}.$$

Compute-optimal training balances parameters and tokens; not simply largest model on fixed data. **Prompting**: zero-shot/few-shot describes task in context; no parameter update. **Instruction tuning**: supervised fine-tuning on instruction-response mixtures.

7.2 RLHF and Hallucination

SFT: next-token imitation on human demonstrations. It cannot express preference among multiple valid answers. **Reward model**: train from pairwise rankings:

$$L_{\text{RM}} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)).$$

Policy optimization with KL:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r_\phi(x, y)] - \beta \text{KL}(\pi(\cdot|x) \| \pi_{\text{ref}}(\cdot|x)).$$

KL prevents policy from drifting too far from SFT/reference. RLHF can reward uncertainty awareness, e.g. "I don't know" when question is unanswerable. Negative samples provide learning signal unavailable in pure SFT.

7.3 RAG and Reasoning

RAG: retrieve documents d then generate:

$$p(y|x) \approx \sum_{d \in \mathcal{R}(x)} p_\theta(y|x, d) p(d|x).$$

Retrieval supplies missing knowledge but does not automatically solve multi-step reasoning. **Inference-time scaling**: spend more test-time compute on search, verification, self-consistency, or long reasoning trajectories. For reasoning model:

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi(\cdot|x)} [R(\text{answer}(\tau), y)] - \beta \text{KL}(\pi \| \pi_{\text{ref}}).$$

Answer verifier gives sparse reward; RL can optimize thinking trajectory even without unique chain-of-thought labels.

7.4 Multimodal Models

Vision-language few-shot: convert image/video tasks to text generation conditioned on visual tokens. **Flamingo**: frozen vision encoder + frozen LM with cross-attention layers and Perceiver resampler for interleaved image/video-text. **LLaVA**: CLIP vision encoder + projection to Vicuna/LLM; visual instruction tuning teaches chat-style responses. **Unified multimodal model**: one decoder handles understanding, generation, editing, reasoning; may use separate experts/tokenizers for text vs image. **BAGEL-style idea**: decoder-only unified model with modality-specific experts; understanding often emerges before reasoning/editing, which need larger scale.

8 Cross-Lecture Map

Density models: EBM defines $p \propto e^{-E}$ but sampling is hard; flow gives exact likelihood via invertible map; VAE gives ELBO; diffusion learns reverse denoising; flow matching learns vector field directly. **Implicit models**: GAN learns sampler without tractable likelihood. **Sequence models**: Transformer turns all modalities into token sequences. **Large models**: pretraining builds capability, post-training aligns behavior, inference-time compute and multimodal grounding extend usability.